

Federated transfer learning-based intrusion detection system in 5G networks

Andrea Bellmunt^{a,1}, Beatriz Otero^{b,1,*}, Eva Rodríguez^{b,1}, Xavier Masip-Bruin^{b,1}

^a Facultat d'Informàtica de Barcelona, Universitat Politècnica de Catalunya (UPC), Campus Nord, Barcelona, Spain

^b Centre de Recerca d'Arquitectures Avançades de Xarxes (CRAAX), Universitat Politècnica de Catalunya (UPC), Vilanova i la Geltru, Barcelona, Spain

ARTICLE INFO

Keywords:

Intrusion detection system
Federated learning
Federated transfer learning
Convolutional neural network
Internet of things
UNSW-NB15 dataset
Bot-IoT dataset
Weight sharing
Robust aggregation functions
Geometric median

ABSTRACT

The development of Intrusion Detection Systems (IDS) for the Internet of Things (IoT) and 5G networks is rapidly advancing. This study investigates the application of federated architectures to train detection models while preserving data privacy by eliminating the need for data sharing among devices. We propose a Federated Transfer Learning (FTL) model tailored for scenarios with unbalanced nodes, enhancing the detection capabilities for unknown attacks compared to conventional Federated Learning (FL) approaches. Utilizing the Bot-IoT dataset as the source domain and the UNSW-NB15 dataset as the target domain, our experiments reveal significant improvements in detection performance.

Specifically, nodes characterized by lower proportions of malicious traffic demonstrate up to a 62.614% enhancement in detecting unknown attacks, increasing detection rates from 19.090% to 81.704%. Moreover, our findings indicate that FTL not only improves the identification of unknown threats but also maintains robust performance in detecting both attacks and benign traffic. Notably, the minimum accuracy achieved by the most imbalanced node reaches 0.912, in contrast to 0.741 with standard FL models.

These results highlight the potential of FTL to train robust models across distributed nodes while ensuring privacy, thereby contributing to improved security measures in IoT and 5G networks.

1. Introduction

While the fourth generation of Wireless Cellular Technology (4G) remains prevalent, it exhibits certain limitations, as noted by Chetri and Bera (2020). To address these shortcomings, the fifth generation (5G) is becoming increasingly established, enabling advanced functionalities, including the Internet of Things (IoT) and other complex applications. Given its growing prevalence, ensuring the safety of 5G networks is crucial. Therefore, the implementation of Intrusion Detection Systems (IDSs) and other security methods is being extensively investigated.

This article focuses on the development of a Network Intrusion Detection System (NIDS) employs Deep Learning techniques. Specifically, the main goal is to implement a federated architecture to simulate distributed real-life scenarios. The focus on Federated Transfer Learning (FTL) seeks to enhance the detection capabilities for unknown attacks compared to Federated Learning (FL) models. This approach considers

local nodes with heterogeneous data distributions, allowing an examination of how the presence of malicious observations during local training affects detection performance.

A critical challenge in such federated environments is the presence of unbalanced nodes, where some devices possess limited or highly skewed samples of malicious traffic. This data heterogeneity can diminish the effectiveness of traditional FL models, which typically assume more balanced local datasets. To overcome this, the proposed FTL framework incorporates transfer learning techniques within the federated setting, facilitating knowledge transfer from nodes with richer malicious data to those with scarce or imbalanced datasets. This mechanism enables all nodes to improve their detection capabilities, particularly in identifying unknown attacks, thus enhancing the robustness and overall performance of the global intrusion detection system.

This study is part of a broader project (Bellmunt, 2024), based on the findings of Rodríguez et al. (2022). That work focused on developing an IDS for detecting attacks in IoT networks using a centralized

* Corresponding author.

E-mail addresses: andrea.bellmunt@estudiantat.upc.edu (A. Bellmunt), beatriz.otero@ac.upc.edu (B. Otero), eva.rodriguez@upc.edu (E. Rodríguez), xavier.masip@upc.edu (X. Masip-Bruin).

¹ These authors contributed equally to this work.

<https://doi.org/10.1016/j.eswa.2025.130868>

Received 14 April 2025; Received in revised form 10 December 2025; Accepted 14 December 2025

Available online 17 December 2025

0957-4174/© 2025 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Convolutional Neural Network (CNN) model, evaluating both approaches with and without Transfer Learning (TL). The analysis emphasized the performance of the model in detecting both known and unknown attacks. However, it did not account for the distributed nature of IoT devices or privacy concerns related to the sharing of local data during centralized model training. To address these issues, the current project introduces modifications to the training architecture using federated methods.

Currently, existing public datasets that feature attacks in IoT networks are constructed using various simulations. Besides this limitation, each available dataset includes different classes of attacks with unique distributions. They generally vary in size and may contain varying amounts of benign traffic. In this project, the Bot-IoT dataset (Koroniotis et al., 2018b, Koroniotis et al., 2018a, Koroniotis et al., 2020b, Koroniotis & Moustafa, 2020, Koroniotis et al., 2020a, Koroniotis, 2020) and the UNSW-NB15 dataset (Moustafa et al., 2017; Moustafa & Slay, 2015, 2016; Moustafa et al., 2019; Sarhan et al., 2021) are the ones being utilized.

In conclusion, the main contributions of this article are the following:

1. We present a framework for a FTL-based IDS specifically designed to identify attacks in IoT networks. This system utilizes a pre-trained FL model that has been trained using subsets of the Bot-IoT dataset, along with subsets of the UNSW-NB15 dataset, which serve as the target local datasets.
2. We conduct a comparative analysis of the FTL-based detection model against a FL-based detection model that employs partitions of the UNSW-NB15 dataset. This comparison evaluates the models' performances in detecting both benign and malicious traffic, with a particular emphasis on the detection of unknown attacks.
3. We investigate the performance of local models with varying data distributions, specifically examining how the representation of malicious samples in local datasets influences attack detection in the context of FL.
4. Our approach enhances attack detection capabilities on nodes that possess a limited number of attack records, thereby providing a more accurate reflection of real-world commercial applications.

The remainder of the paper is organized as follows: Section 2 describes the related work. Section 3 describes the theoretical concepts behind the article. Section 4 describes the datasets and dataset partitions, preprocessing, IDS and model architectures, development environment, and aggregation functions explored. Section 6 presents the experiments and the analysis of the results. Finally, Section 8 concludes the paper.

2. Related work

This section reviews recent research focusing on the integration of Federated Transfer Learning (FTL) with Intrusion Detection Systems (IDS) in 5G networks. The advent of 5G technology, with its high-speed, low-latency capabilities and massive connectivity, has introduced new security challenges that traditional centralized approaches struggle to address. Initially, FL-based IDS have emerged as a promising solution, enabling distributed and privacy-preserving model training across network edge devices without sharing raw data. Although existing surveys (Blika et al., 2025, Lee et al., 2024, Agrawal et al., 2022) highlight substantial progress in the field, they also emphasize that significant research gaps remain, particularly concerning FL-related challenges. One such challenge is the detection of zero-day attacks, an area where FTL has shown potential. Early works in this area proposed an intrusion detection framework, so-called IoTDefender (Fan et al., 2020), leveraging FTL. It employed customized detection models tailored to different IoT networks, achieving an average detection accuracy of 91.93% while maintaining data privacy and confidentiality. The framework involved four clients, each training local models using their own private datasets. The CICIDS2017 dataset (Sharafaldin et al., 2018) was initially used to train a global model using a Convolutional Neural Network

(CNN), after which each client refined their local models with their own data, enhancing the framework's adaptability and performance. Subsequent works focused on specific scenarios as the Internet of Medical Things (IoMT) or the industrial IoT (IIoT). The authors in Otoum et al. (2022) examined the effectiveness of integrating FL and TL techniques with various ML and DL algorithms to improve IDS to protect IoT applications. The Internet of Medical Things (IoMT) was used as a case study to demonstrate how model performance is improved by accelerating the learning process, reducing the volume of data required for training, and ensuring user data privacy. Alharbi (2024) continues this work to improve the model by enabling clients to obtain a personalized model while benefiting from the knowledge of the other clients of the framework. This model achieves better results in the detection of unknown attacks achieving detection accuracies by 95%-99%. In the Industrial Internet of Things (IIoT), an FTL based IDS is proposed (Zhang et al., 2022), enabling privacy-preserving machine learning for future IIoT networks. The solution used instance-based transfer learning at the local level to address the challenge of training local models with non-independent and identically distributed (non-IID) data. The solution makes use of a rank aggregation algorithm combined with a weighted voting mechanism, achieving a detection accuracy by 95.97% for AdaBoost and 73.70% for Random Forest. In the same area, Verma et al. (2023) proposes a FTL based IDS that makes use of a dual Autoencoder (AE) model that enables efficient collaboration and knowledge sharing among distributed nodes, resulting in improved model generalization and scalability. This approach achieves accuracy rates of 99.32%. In contrast to our work, which targets broader applications, their study focuses on the Industrial Internet of Things and utilizes the X-IIoTID dataset. This dataset is limited in scope, being specifically designed for IIoT scenarios and containing a relatively small number of traces, particularly for cyberattack instances. Finally, recent work in 5G networks uses FL to improve privacy protection (Alalyan et al., 2024) and to secure network slices and detect potential attacks (Djaidja et al., 2024). In contrast to prior work, this study proposes a Federated Transfer Learning (FTL) framework that unifies federated learning with personalized fine-tuning. Instead of applying FL and TL sequentially or within homogeneous architectures, our method adapts a pre-trained source-domain model individually at each target node via fine-tuning, without an additional aggregation step. This approach effectively handles highly imbalanced distributions and scarce malicious traffic, improving detection of unknown attacks compared to previous FTL methods. Furthermore, by combining knowledge transfer and model personalization into a single process, our framework differs from solutions like IoTDefender, which rely on less heterogeneous architectures or less challenging domain distributions.

Table 1 provides a summary of the research surveyed on FTL-based IDS in 5G networks, as well as in specific environments such as the IoMT and IIoT.

Our work differentiates itself from previous research work in several key ways:

- It does not limit its focus to specific environments, such as IIoT or IoMT.
- It addresses the detection of zero-day attacks.
- It tackles the performance challenges present in local models that arise from an imbalance between benign and malicious samples.

In this way, our proposed system achieves significant improvements in detection accuracy, with rates of up to 99% for known attacks and 91.93% for zero-day attacks.

This advancement underscores the potential of our approach to enhance the robustness and effectiveness of FTL-based IDS across a wide range of applications and network scenarios.

Table 1
Comparison of intrusion detection systems based on FTL techniques.

Reference	ML/DL model	Dataset	Domain	Zero-day attacks	Clients	ACC
Fan et al. (2020)	CNN	CICIDS2017 NSL-KDD IOT	5G IoT	Yes	3	91.93 %
Otoum et al. (2022)	CNN DNN	Edge-IIoTset	IoMT	No	4	99.3 %
Alharbi (2024)	DNN	CIC-ToN-IoT	IoMT	Yes	4	95 %-99 %
Zhang et al. (2022)	AdaBoost Random Forest	CICIDS2017 CICIDS2018	IIoT	No	5	73.70 % - 95.97 %
Verma et al. (2023)	Dual AE	X-IIoTID	IIoT	Yes	2–20	99.3 %
Proposed work	CNN	Bot-IoT UNSW-NB15	5G IoT	Yes	5	81 % - 99 %

3. Theoretical framework

3.1. 5G and IoT networks

5G, the latest generation of wireless communication technology, provides enhanced reliability, higher data rates, and improved connectivity compared to its predecessors (Gupta & Jha, 2015). This advancement has accelerated the integration of a wide range of new devices and enabled the deployment of more demanding applications, particularly within the Internet of Things (IoT) ecosystem.

The IoT encompasses a large network of distributed physical devices capable of collecting, exchanging, and processing data to optimize and automate various processes (Madakam et al., 2015). Using locally generated data, organizations can develop models that support advanced decision-making, offering substantial potential across multiple domains, most notably in healthcare, where data from wearable technologies can provide invaluable insights.

Multiple public datasets related to 5G and IoT networks are available. This project utilizes two prominent datasets: the Bot-IoT dataset (Koroniotis et al., 2018b) and the UNSW-NB15 dataset (Sarhan et al., 2021). Other notable ones include CICIDS2017 (Sharafaldin et al., 2018) and 5G-NIDD (Samarakoo n et al., 2022).

3.2. Intrusion detection system

Intrusion Detection Systems (IDSs) play a crucial role in identifying malicious activities by differentiating them from legitimate behavior (Rodríguez et al., 2021). They primarily employ two detection methodologies: signature-based and anomaly-based detection. Signature-based systems match observed activities with known patterns, limiting their ability to detect unknown attacks. In contrast, anomaly-based IDSs focus on identifying zero-day attacks by analyzing behavioral patterns, which is the focus of this paper.

While traditional approaches have utilized machine learning (ML), recent research is shifting towards deep learning (DL) models to enhance detection rates. IDSs can also be classified by the monitored environment. Host-based IDSs operate on individual hosts, alerting users to detected issues, while Network Intrusion Detection Systems (NIDS) monitor network traffic from strategic points, such as gateways, analyzing inbound and outbound packets without impacting legitimate traffic.

3.3. Convolutional neural network

A Convolutional Neural Network (CNN) is a type of Deep Learning model commonly used for pattern recognition and feature extraction tasks, particularly in image processing. However, CNNs have proven effective across various domains, such as attack detection, due to their ability to learn complex patterns from data (Mohammadpour et al., 2022).

The architecture of a CNN, illustrated in Fig. 1, typically consists of three main types of layers:

- **Convolution Layers:** They perform convolution operations between the input data and filters (kernels) to extract local features.
- **Pooling Layers:** By down-sampling the feature maps, they reduce dimensionality and help prevent overfitting.

- **Fully Connected Layers:** They connect each neuron of a layer to each neuron of the next, integrating extracted features to perform the final output.

In addition, CNNs generally incorporate activation functions to include non-linearity. The Rectified Linear Unit or ReLU is one of them and is defined as:

$$ReLU(x) = \max(0, x) \quad (1)$$

where x represents the output of a convolution layer or any preceding layer in the architecture.

For classification tasks, the Softmax function is usually used in the output layer to convert raw prediction scores into class probabilities, as follows:

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (2)$$

z_i represents the i -th element of the input vector z . z contains the raw prediction scores for each class. The term $\sum_{j=1}^K e^{z_j}$ is the sum of the exponentials of all the elements in z , and K is the total number of classes.

3.4. Federated learning

Federated Learning was first introduced by Google nearly a decade ago as a novel approach to enable collaborative model training across distributed devices or systems (McMahan & Ramage, 2017). The main idea was to avoid transmitting private data by sharing only model updates from each device. These updates, obtained by training a global model locally on private data, would be securely aggregated on a central coordinating server. The global model would then be iteratively updated with these aggregated updates. That is, Federated Learning was presented to prioritize the preservation of data privacy, ensuring that raw data remained on the devices originating.

Over the years, the idea has expanded significantly, addressing diverse challenges and applications. The original formulation of FL is now categorized as Horizontal Federated Learning (HFL) (Xia et al., 2022). This involves datasets across nodes with a common feature space, but containing different samples. Such scenarios usually happen with devices within the same company, considering that they observe the same variables of interest.

On the other hand, Vertical Federated Learning (VFL) enables collaborative learning by using heterogeneous datasets with different feature spaces. This approach is applicable when datasets represent the same group of subjects, but vary in the features they contain. VFL allows multiple parties to collaboratively train models without exposing sensitive information, which is of significant utility in scenarios where data is fragmented between organizations or domains. Both HFL and VFL are represented in Fig. 2.

For clarity, this paper assumes that any mention of Federated Learning refers specifically to Horizontal Federated Learning unless otherwise stated.

3.5. Federated transfer learning

Although it has not been mentioned in the previous section, there is another type of FL which integrates Transfer Learning (TL).

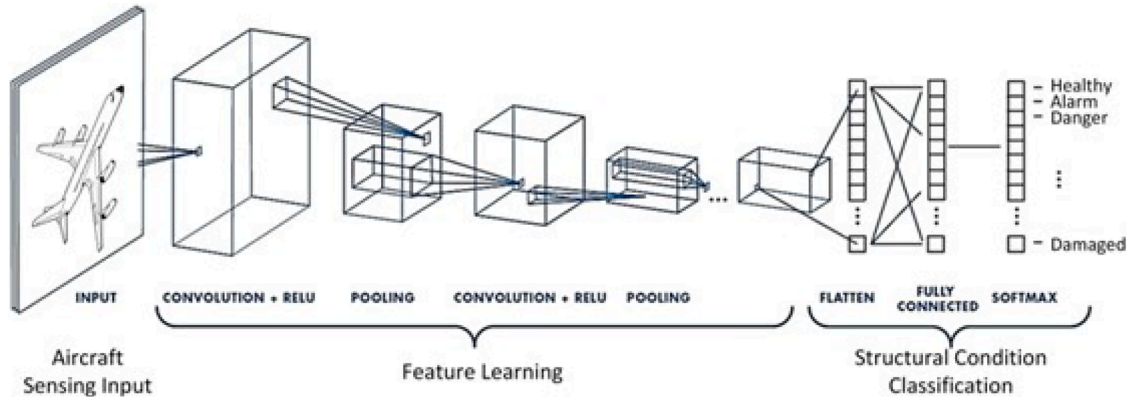


Fig. 1. Architecture of a convolutional neural network. source: Tabian et al. (2019).

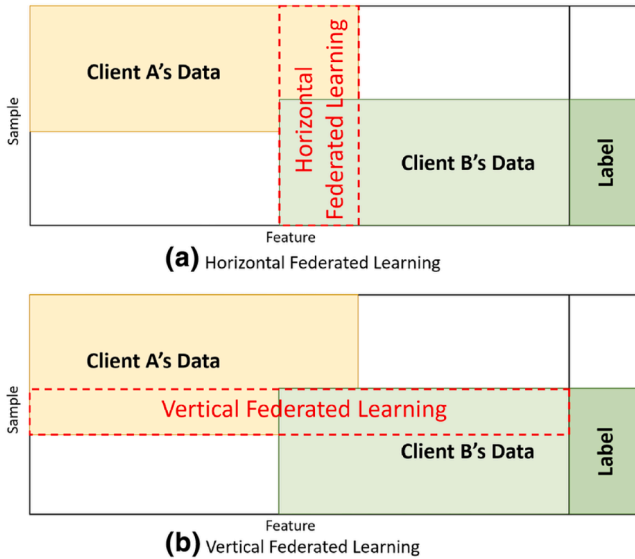


Fig. 2. Conceptual diagram of horizontal and vertical federated learning. image adapted from (Xia et al., 2022).

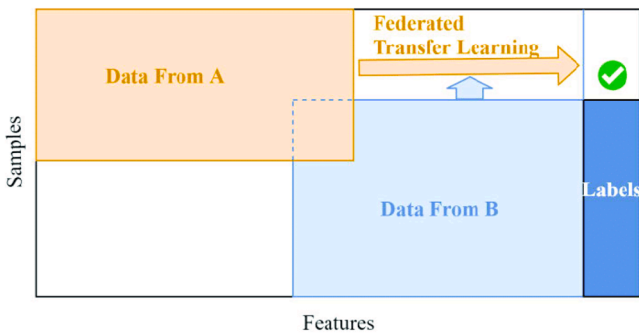


Fig. 3. Overview of the federated transfer learning framework (adapted from Rahman et al., 2021).

Federated Transfer Learning (FTL) (Rahman et al., 2021) is particularly useful when the source and target domains have few overlapping features and samples, as shown in Fig. 3. However, it can also be applied when the domains have a greater degree of feature overlap. FTL facilitates knowledge transfer in a federated context, generally enhancing the model's performance on the target task.

For additional context, TL is an ML technique in which a pre-trained model, called the source, is used to learn related tasks with new datasets

(Iman et al., 2023). One common way to implement TL is through Fine-Tuning. Specifically, Feature Extraction Fine-Tuning involves using the pre-trained model by freezing its initial layers, which are not retrained. These earlier layers typically capture generic features that are broadly applicable across different tasks.

In this process, the later layers of the pre-trained model, which capture task-specific features, are either replaced with new layers or retained but retrained. These new or modified layers are then trained on the target domain, which represents the dataset associated with the new specific task.

In summary, TL involves a source and a target domain. The source domain is used to train the initial model, and the target domain is incorporated during the Fine-Tuning process, where the pre-trained model is adapted and optimized for specific requirements of the target task.

4. Methodology

4.1. Development environment

The main experiments have been developed using an ASUS UX430U Notebook PC, the processor of which is the Intel(R) Core(TM) i7-8550U CPU @ 1.80 GHz 1.99 GHz. It offers 16 GB of installed RAM.

As for the federated simulations, they have been created using Python3 v3.10.12, with libraries TensorFlow v2.11.0 (Abadi et al., 2016), using Keras from it, scikit-learn v1.5.2 (Pedregosa et al., 2011), Pandas v2.2.3 (McKinney et al., 2010), and Numpy v.1.25.2 (Harris et al., 2020). The complete implementation is publicly available on our GitHub repository (Bellmunt, 2025).

4.2. IDS and model characteristics

The purpose of this section is to present a clear and thorough explanation of the IDS's structural configuration and its federated approach. It also offers a detailed analysis of the detection model's architectural framework, covering both FL and FTL components.

The IDS being developed follows the pipeline illustrated in Fig. 4. In theory, the system first captures network traffic using sensors; however, in our case, the traffic is sourced from the selected datasets. Once the sample or a set of samples has been collected, the next step is to preprocess the data, preparing them for input into the detection model. The model is then applied to distinguish between benign and malicious traffic. Additional steps, such as attack classification, may follow the detection process to further analyze and categorize the identified threats.

Focusing on the detection task, the models are trained using FL or FTL. To clarify the distinction between the two, the specific training algorithms are defined. Although the data collection and preprocessing steps that precede the training process will be explained in more detail in

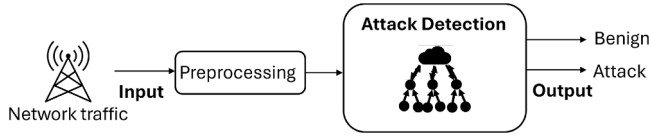


Fig. 4. Intrusion detection system workflow.

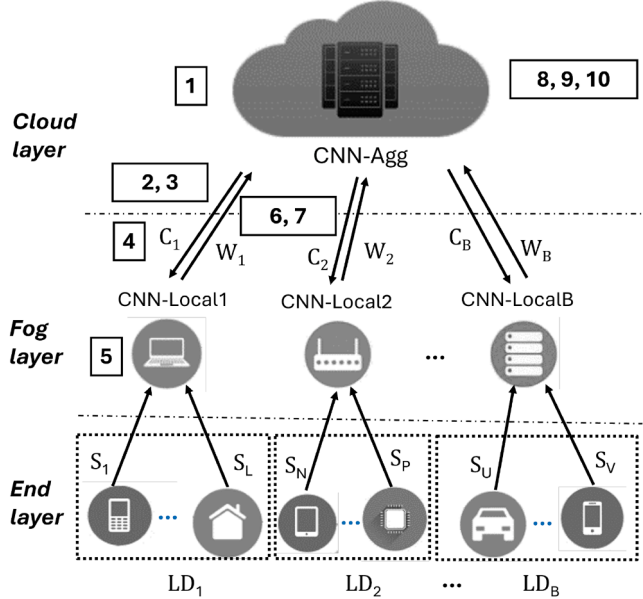


Fig. 5. Generalization of the federated learning (FL) framework.

the following section, it is important to note that these tasks are carried out prior to applying the algorithms presented here.

The first algorithm to consider is the FL algorithm, shown in Fig. 1. The core concept is to continuously train and aggregate a predefined set of local models until convergence is achieved, using a shared global model as the initial point for local training.

To provide a clearer understanding of the FL process, Fig. 5 illustrates the layers and components involved in these tasks. Three layers contribute to the final global model training. Starting from the bottom, the first is the “End Layer”, which consists of all the devices or end-nodes where the network traffic is collected. These traffic samples are organized into local datasets (LD). Each fog node, located in the “Fog Layer”, is responsible for accessing a specific local dataset, LD_b . The fog nodes serve as computational servers where the local models are trained. In this project, $B = 5$, meaning that only five nodes are included in the FL process.

For clarity, from this point forward, the term “node” refers to the combination of a set of end-nodes and their associated fog-node. In other words, each node trains a local model using its corresponding local dataset.

The third and top layer, known as the “Cloud Layer”, contains the global model ($CNN - Agg$), which is produced by aggregating all the local models. Once a fog-node gains access to its local dataset (LD_b), the training process begins by obtaining a copy of the global model (C_b). The local model ($CNN - Localb$) is then trained using this copy and the local dataset. Upon completion, the trained model’s weights are sent to the Cloud, where they are aggregated to update the global model ($CNN - Agg$). This updated global model can be used in the next iteration of the training. The number of global iterations (I) is defined as a hyperparameter. To support the understanding of key terms, Table 2 describes each variable used in Algorithm 1, which outlines the step-by-step procedure. Finally, Fig. 6 presents a schematic representation of the complete Federated Learning (FL) process described above.

Table 2

Definitions of variables and parameters in Algorithm 1.

Variable	Description
E	Local epochs
b	Node ID
W_G	Global model weights
W_b	Local model weights
I	Global iterations
LD_b	Node b local dataset
$CNN - Agg$	Aggregated global model
$CNN - Localb$	Node b local model
C_b	Global model copy

Algorithm 1 Federated learning.

- 1: Initialization of Global Model $CNN - Agg$ with weights W_G
- 2: **for each** Global Iteration I **do**
- 3: **for each** Local Node b **do**
- 4: Initialization of Local Model $CNN - Localb$ from Copy of Global Model C_b with weights W_G
- 5: Training of $CNN - Localb$ with Local Dataset LD_b for E epochs
- 6: Send the new W_b to the Cloud layer
- 7: **end for**
- 8: **end for**
- 9: Evaluate $CNN - Agg$ performance

Table 3

Definitions of the new variables and parameters in Algorithm 2.

Variable	Description
FT_b	Fine-tuning process for node b
$CNN - Localb - TL$	Node-tailored local model via fine-tuning

Algorithm 2 Federated transfer learning.

Require: Pre-trained model with frozen layers *Fixed* $CNN - Agg$

- 1: Initialization of Global Model as *Fixed* $CNN - Agg$
- 2: **for each** Local Node b **do**
- 3: Get Copy of Global Model C_b
- 4: Apply proper Fine-Tuning FT_b to get $CNN - Localb - TL$ initialization
- 5: Training of $CNN - Localb - TL$ with Local Dataset LD_b for E epochs
- 6: Evaluate $CNN - Localb - TL$ performance
- 7: **end for**

In contrast, FTL introduces several modifications to the structure. As outlined in Algorithm 2, the goal is not to update an aggregated global model but to fine-tune local models, enhancing their individual performance. As shown in Fig. 7, the “End Layer” remains unchanged. The “Cloud Layer” is similar to that in FL, but instead of starting with a random global model, it contains a pre-trained model (*Fixed* $CNN - Agg$) prior to the first iteration.

The “Fog Layer” undergoes the most significant changes. Training begins once the local dataset (LD_b) is available and after a copy of the pre-trained model has undergone a specific fine-tuning process (FT_b). This fine-tuned model is then trained using the local data. However, unlike FL, the resulting model is not aggregated with others after training. Instead, it remains focused on improving performance locally.

In this case, model aggregation could be useful when all local models are being refined to solve the same task and share a common architecture. However, when nodes differ significantly from each other, universal fine-tuning may not be suitable for every node. Therefore, we opt to personalize each fine-tuning process (FT_b) to the specific needs of each node, resulting in multiple distinct models ($CNN - Localb - TL$). Due to this individualized approach, no further aggregation steps are

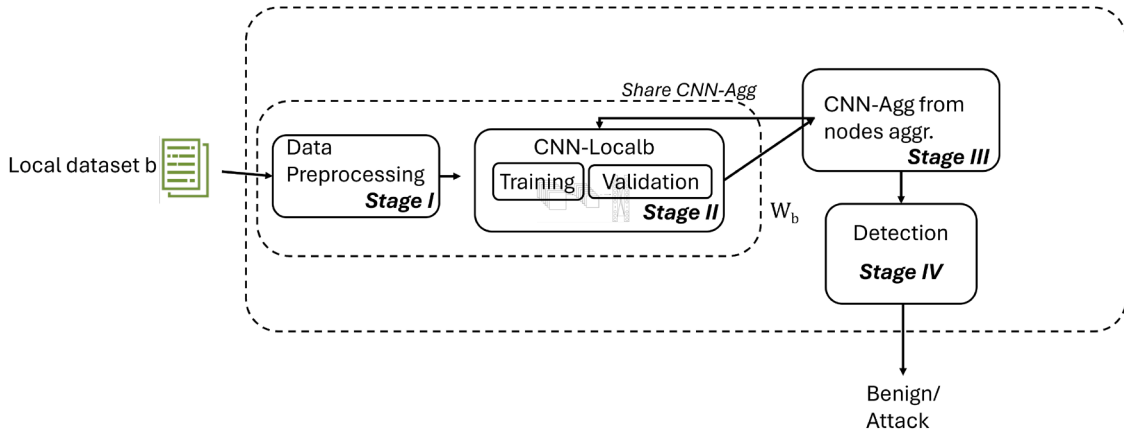


Fig. 6. Overview of the federated learning (FL) experimentation process.

incorporated into the process. As in Algorithm 1, Table 3 provides descriptions of the new variables used in Algorithm 2. Finally, Fig. 8 presents a comprehensive workflow of the entire process outlined previously for the FTL model.

It is important to highlight the fundamental differences between the proposed FTL approach and the conventional FL scheme. First, during the fine-tuning process, the initial convolutional layers of the pre-trained model are frozen, thereby preserving the generic representations learned from the source domain. Only the final dense layers are adjusted at each node to capture patterns specific to the target domain. Second, after individual fine-tuning, the local models are not aggregated again; each node retains its personalized model. This design choice is motivated by the heterogeneity of local distributions and the need to optimize detection in nodes with scarce malicious samples, avoiding the loss of relevant information that could occur when merging highly imbalanced models. Collectively, these characteristics distinguish our approach from the classical FL paradigm, providing a flexible mechanism adapted to realistic traffic scenarios.

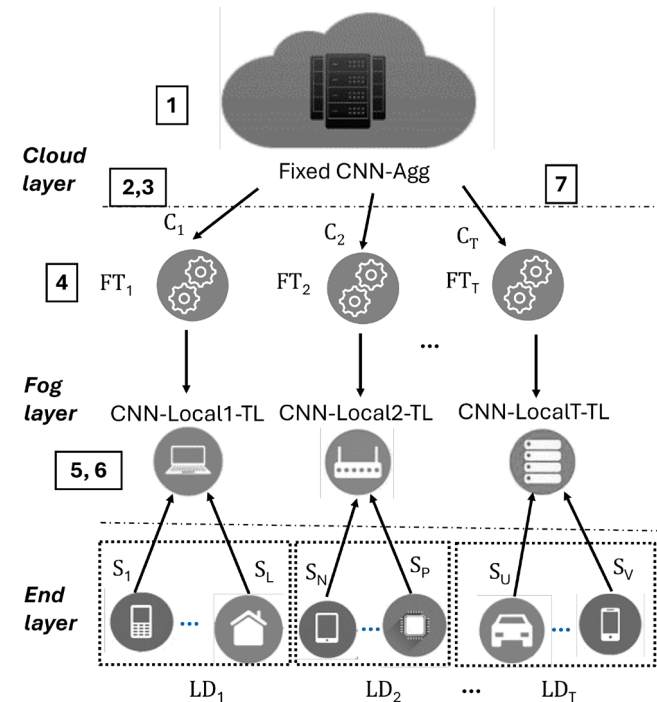


Fig. 7. Generalization of the federated transfer learning (FTL) framework.

4.3. Datasets and partitions

The implementation of FTL requires two distinct datasets, identified as the source and target domains. This article builds on the selection and preprocessing of datasets established in the work of Rodríguez et al. (2022). Accordingly, the Bot-IoT dataset serves as the source domain, while the UNSW-NB15 dataset acts as the target domain. Both datasets are publicly available and were produced by UNSW Canberra.

The original Bot-IoT dataset comprises approximately 72 million samples, the majority of which consist of malicious traffic. For model training, a small percentage of these samples is used to achieve a balance between benign and attack observations. This dataset contains 44 variables and the label, with attack types including: *Normal*, *DoS*, *DDoS*, *Reconnaissance* and *Theft*.

In contrast, the UNSW-NB15 dataset features around 2.5 million samples and includes 49 available variables. Unlike the Bot-IoT dataset, most of the samples in UNSW-NB15 are benign. The attack categories represented include: *Normal*, *Generic*, *Exploits*, *DoS*, *Reconnaissance*, *Fuzzers*, *Analysis*, *Backdoors*, *Shellcode* and *Worms*, although not all categories are equally represented.

For the transfer learning process, a subset of variables is selected for training based on their overlap between the datasets and their significance. This subset consists of 13 variables that include elements such as transaction protocol, IP addresses, port numbers, total duration, and packet count, along with a Boolean label indicating whether an observation is benign or malicious.

The federated framework implemented in this study consists of five distinct nodes, which requires partitioning the selected datasets into five segments. The decision to use five nodes is based on the amount of data available after preprocessing. This approach seeks to maximize the number of local nodes while ensuring that each local dataset has enough data for local training.

This section outlines the variations included in the experiments, categorizing them into detection variations from the UNSW-NB15 dataset and detection partitions from the Bot-IoT dataset.

4.3.1. UNSW-NB15 partitioning

Although in Bellmunt (2024) several variations of the partitions dataset are generated, in this case we are only interested in the so-called DET-IMBALANCED3 variation, which presents unbalanced nodes with different benign-malicious proportions. These partitions were obtained using an undersampling technique. In contrast, other related works, such as Jamal et al. (2023a), Jamal et al. (2023b) and Jamal et al. (2025), explore alternative strategies for addressing class imbalance, employing various oversampling methods.

In terms of training, validation, and test partitions, 20% of the samples in all training variations are allocated for validation purposes. Then,

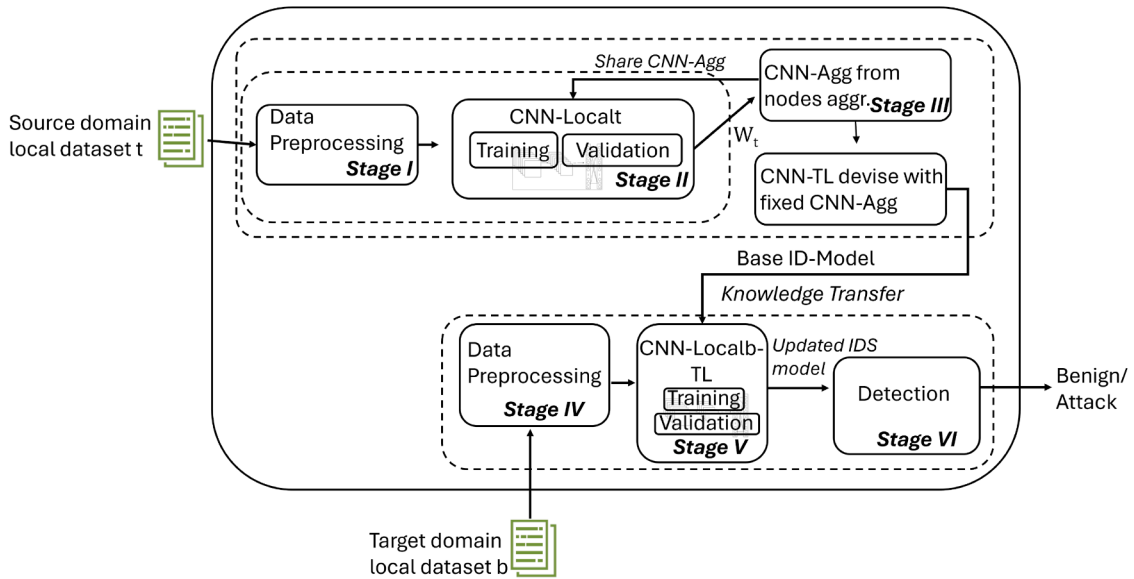


Fig. 8. Overview of the federated transfer learning (FTL) experimentation process.

Table 4

Distribution of known attack samples across DET-IMBALANCED3 and DET-TEST dataset partitions.

ID	Benign	Generic	Exploits	DoS	Reconnaissance	Total
1	31,480	24,840	4274	583	1783	62960
2	37,776	18,544	4274	583	1783	62,960
3	47,200	9095	4274	583	1782	62,934
4	56,682	1955	1998	583	1782	63,000
5	44,082	12,281	4274	582	1782	63,001
DET-IMB3	217,220	66,715	19,094	2914	8912	314,855
DET-TEST	72,542	53,648	6944	949	2941	137,024

from the total, around a 30 % of the observations are used for the test dataset (34 % in the case of DET-IMBALANCED3).

DET-IMBALANCED3, the training dataset, is built to be able to observe how varying proportions of benign and malicious network traffic can impact both local and global models. Recognizing that real-world scenarios often feature a higher prevalence of benign traffic, none of the nodes in this variation exhibit a majority of malicious samples.

Node 1 features a balanced representation of both benign and malicious network traffic. Node 2 contains 10 % more benign samples than the first one, resulting in a proportion of 60 % benign and 40 % attacks. Then, node 3 has a distribution of 75 % benign cases and 25 % attacks, while Node 5 has slightly fewer benign samples in a 70/30 ratio. Finally, Node 4 illustrates a more pronounced imbalance, with 90 % of its cases being benign. The resulting distributions are depicted in Table 4.

As for DET-TEST, it comprises a subset of samples designated for testing purposes in both Federated Learning (FL) and Federated Transfer Learning (FTL) scenarios. This subset has been carefully constructed to ensure appropriate relevance for each type of sample, which includes benign traffic, known attacks, and unknown attacks, as seen in Table 4. The relevance is determined more by the significance of their detection than by their frequency in real-world scenarios.

In this dataset, benign samples predominate, accounting for 45 % of the total. This is followed by known attacks, which constitute 40 % of the samples. Unknown attacks, representing 15 % of DET-TEST, include categories that are not utilized in training or validation, reflecting threats that have not yet been publicly documented. These categories encompass *Fuzzers* (21516), *Shellcode* (1511), *Analysis* (622), *Backdoors* (357), and *Worms* (174), giving a total of 24180 unknown attacks in DET-TEST, and 161204 total samples counting unknown attacks.

Table 5

Distribution of samples across BOT-UNIFORM and BOT-TEST dataset partitions.

ID	Benign	DDoS	Theft	DoS	Reconnaissance	Total
BOT-UNIF-1	1407	1510	241	1501	1495	6154
BOT-UNIF-2	1407	1510	241	1501	1495	6154
BOT-UNIF-3	1407	1510	241	1501	1495	6154
BOT-UNIF-4	1407	1510	240	1502	1495	6154
BOT-UNIF-5	1411	1467	240	1546	1495	6159
BOT-UNIF	7039	7507	1203	7551	7475	30775
BOT-TEST	2407	2493	384	2449	2525	10,258

4.3.2. Bot-IoT partitioning

Now, in terms of the source domain dataset, the selected Bot-IoT samples are divided into 75 % for training and validation purposes, the remaining 25 % reserved for testing. The validation set constitutes 20 % of the training observations.

The test dataset, called BOT-TEST, maintains a distribution similar to that of the five training partitions, as illustrated in Table 5. The rationale behind uniformly distributing samples across nodes is to optimize the performance of the pre-trained model.

In terms of sample sizes, all nodes contain approximately 6,154 samples. This contrasts with the UNSW-NB15 dataset, where the number of samples is significantly higher. The primary reason for this discrepancy is that the original Bot-IoT dataset contains a similar number of benign samples prior to preprocessing. After preprocessing, only 9,446 cases remain. If an excessive amount of attack traffic was included, it could adversely affect the model's performance, leading to a tendency to predict an attack in every instance.

4.4. Aggregation functions

Federated architectures rely on the aggregation of multiple local models, and the aggregation method can significantly affect the performance of the global model. To select the aggregation function used in our models, we conducted a study that includes four of the most commonly used functions: FedAVG (McMahan et al., 2017), Median (Li et al., 2024), Krum (Blanchard et al., 2017), and Geometric Median (Pillutla et al., 2022). Of these, the last three are designed to mitigate the effects of outliers or corrupted local models on the global model. The study assesses the effectiveness of each metric in IDS scenarios using

Table 6
Confusion matrix for the classification results.

		Predicted Value	
		Positive	Negative
Actual Value	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

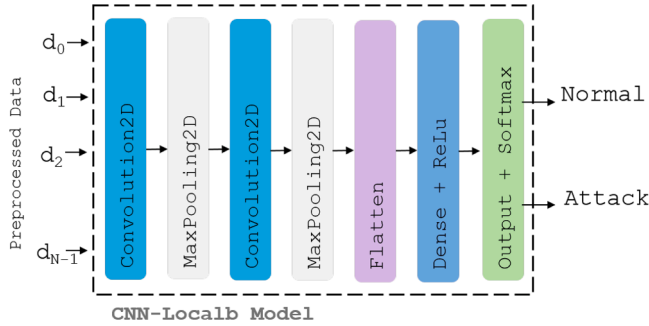


Fig. 9. Convolutional neural network (CNN) architecture for local models in federated learning (FL).

both FL and FTL models, revealing that the Geometric Median proves to be the most effective metric. The details of this study can be found in Bellmunt (2024).

4.5. Evaluation metrics

The performance of the models was evaluated using metrics derived from the confusion matrix (Table 6).

These metrics are defined as follows:

- **False Negative Rate (FNR):** Measures the proportion of positive instances that were incorrectly classified as negative. Formally,

$$FNR = \frac{FN}{TP + FN}.$$

- **Precision:** Quantifies the proportion of predicted positive instances that are actually positive, given by

$$Precision = \frac{TP}{TP + FP}.$$

- **Recall:** Indicates the proportion of actual positive instances that are correctly identified, defined as

$$Recall = \frac{TP}{TP + FN}.$$

- **F1-Score:** Represents the harmonic mean of Precision and Recall, balancing both metrics, calculated as

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}.$$

- **Accuracy:** Reflects the overall proportion of correctly classified instances, calculated by

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

5. Experiments

In this section, we present the architecture and hyperparameters used for each training model discussed in Section 4.2, as illustrated in Figs. 6 and 8.

Fig. 9 shows the architecture of the models trained using FL (*CNN-Localb*) and the pre-trained model in the FTL scenario (*CNN-Localt*), which is also trained using FL.

The characteristics of the trained models are detailed in Table 7. This table specifies the model's input shape ("Inp. Shape"), as determined by

previous preprocessing steps; the number of filters in each convolutional layer ("Filters"); the kernel size ("Kernel"); the pool size of the pooling layers ("Pool"); the number of units in the dense layer ("Dense"); the learning rate ("Lr. Rate"); and the batch size ("Batch").

The local models in FL are trained over five global iterations, with each iteration consisting of one epoch. These hyperparameters were established on the basis of a preliminary study on model convergence detailed in Bellmunt (2024). For the pre-trained model in FTL, the study utilized the BOT-UNIFORM dataset, employing five global iterations with five local epochs each. Early stopping is implemented with a patience of two epochs, monitoring the validation loss.

In our experiments, the communication overhead in FL is relatively moderate, since model weights (approximately a few megabytes) are exchanged between the server and clients at the end of each round. Each global round involves sending and receiving the full model once per node. In the standard FL setup, this configuration of five global rounds with one local epoch per node resulted in a total training time of approximately 2 min and 10 s, including both local computation and communication delays (roughly doubling the per-epoch training time to account for coordination between nodes).

In contrast, the fine-tuned local (FTL) approach first trains the base model for five rounds of five epochs each, followed by node-specific fine-tuning for 20 epochs with early stopping. FTL training takes about 20 s total. In both FL and FTL configurations, the loss and accuracy metrics generally stabilize by the fourth or fifth global round across all nodes. For node-specific tuning strategies, we employed the Keras Tuner library to automatically select hyperparameters such as learning rate and batch size individually for each node, enabling local models to better adapt to node-specific data distributions and computational capacity.

On the other hand, Fig. 10 shows the overall design of *CNN-Localb-TL*. The convolutional layers enclosed within the first dashed box represent the frozen layers, which retain their pre-trained weights. Only the unfrozen layers are fine-tuned. This model is then trained independently with an emphasis on maximizing validation accuracy. The hyperparameters are detailed in Table 8, which shows the configuration for dense layers and the learning rate. The configurations for the two convolutional layers are the same as those for the FL model presented in the previous table. Therefore, Table 8 shows that the "Dense" columns specify the number of units in each of the four dense layers, while the "Drop" columns indicate the dropout rates for the associated dropout layers. Lastly, "Lr. Rate" column presents the learning rates.

All TL model training processes utilize 20 epochs, with early stopping implemented and a patience of 2, while monitoring the validation loss.

6. Results and discussion

We aim to investigate the performance of FL and FTL in cases with different proportions of attack transactions.

To ensure the statistical reliability of the reported results, each experiment was performed 50 times under identical conditions. The average values of the evaluation metrics (FNR, accuracy, precision, recall, F1-score) are presented in Table 9, which compares the local models of Federated Learning (FL) and Federated Transfer Learning (FTL) using the DET-IMBALANCED3 dataset. The corresponding confidence intervals 95% for these metrics are shown in Table 10, estimated from the standard error of the mean. In particular, the mean values reported in Table 9 fall within the corresponding confidence intervals listed in Table 10 for all metrics and experimental scenarios. This alignment demonstrates that the observed results are statistically robust and consistent, suggesting that the reported performance of the FL and FTL local models is reliable and is not significantly influenced by random variation.

From Table 9, several observations can be made. First, when examining the application of FL, the node with benign samples 50% shows exceptional performance in all metrics. The high proportion of attacks enables the local model to generalize effectively and achieve robust detection. Upon increasing benign transactions by 10%, the metrics

Table 7
Hyperparameters of the federated learning (FL) model.

Inp. Shape	Filters0	Kernel0	Pool0	Filters1	Kernel1	Pool1	Dense	Lr. Rate	Batch
(24, 1, 1)	32	(1, 10)	(1, 2)	64	(1, 10)	(1, 2)	444	$5e-4$	2048

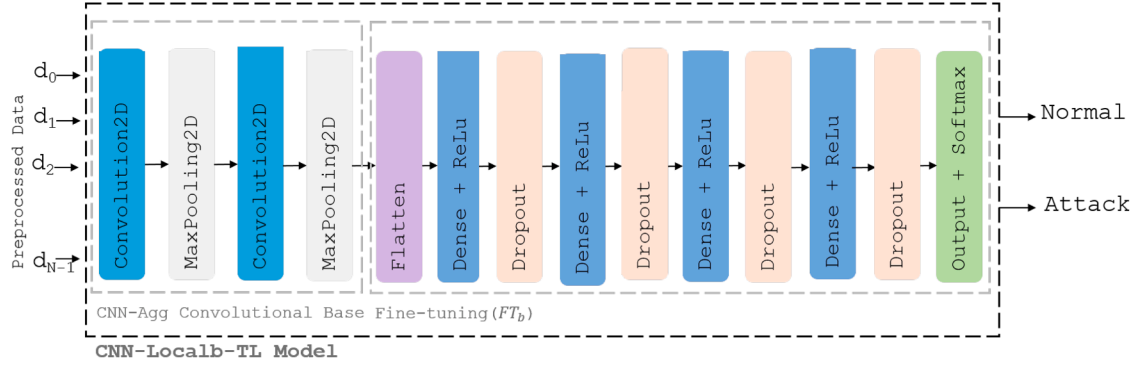


Fig. 10. Fixed pre-trained CNN with fine-tuning generalization for local models in federated transfer learning (FTL).

Table 8
Hyperparameters of the federated transfer learning (FTL) model.

Node ID	Dense0	Drop0	Dense1	Drop1	Dense2	Drop2	Dense3	Drop3	Lr. Rate
1	128	0.3	512	0.4	32	0.4	192	0.3	$9e-4$
2	448	0.3	224	0.3	64	–	256	0.3	$3e-3$
3	224	0.1	256	0.3	128	0.1	256	0.1	$1e-3$
4	480	0.4	160	0.1	480	–	32	0.4	$3e-3$
5	192	0.1	64	0.3	384	0.1	480	0.2	$1e-3$

Table 9
Comparative performance of federated learning (FL) and federated transfer learning (FTL) local models using DET-IMBALANCED3.

Benign %	Method	FNR	F1-score	Precision	Recall	ACC	%Unknown
50	FL	0.001	0.992	0.986	0.999	0.992	99.959
	FTL	0.000	0.993	0.985	1.000	0.992	99.876
60	FL	0.007	0.989	0.986	0.993	0.988	98.614
	FTL	0.001	0.993	0.986	0.999	0.992	99.872
70	FL	0.066	0.961	0.989	0.934	0.958	83.954
	FTL	0.024	0.983	0.989	0.976	0.981	94.218
75	FL	0.115	0.934	0.989	0.885	0.931	71.985
	FTL	0.035	0.976	0.987	0.965	0.974	89.309
90	FL	0.468	0.693	0.993	0.532	0.741	19.090
	FTL	0.147	0.914	0.985	0.853	0.912	81.704

Table 10
Confidence intervals 95 % for evaluation metrics of FL and FTL local models using DET-IMBALANCED3.

Benign %	Method	FNR	F1-score	Precision	Recall	ACC
50	FL	[0.000753, 0.001247]	[0.991, 0.993]	[0.985, 0.987]	[0.9990, 0.9993]	[0.991, 0.993]
	FTL	[0.000000, 0.000001]	[0.992, 0.994]	[0.984, 0.986]	[0.999, 1.002]	[0.991, 0.993]
60	FL	[0.006349, 0.007651]	[0.988, 0.990]	[0.985, 0.987]	[0.992, 0.994]	[0.987, 0.989]
	FTL	[0.000753, 0.001247]	[0.992, 0.994]	[0.985, 0.987]	[0.998, 0.999]	[0.991, 0.993]
70	FL	[0.064061, 0.067939]	[0.959, 0.963]	[0.988, 0.990]	[0.932, 0.936]	[0.956, 0.960]
	FTL	[0.022805, 0.025195]	[0.982, 0.984]	[0.988, 0.990]	[0.975, 0.977]	[0.980, 0.982]
75	FL	[0.112508, 0.117492]	[0.932, 0.936]	[0.988, 0.990]	[0.883, 0.887]	[0.929, 0.933]
	FTL	[0.033564, 0.036436]	[0.975, 0.977]	[0.986, 0.988]	[0.964, 0.966]	[0.973, 0.975]
90	FL	[0.464104, 0.471896]	[0.689, 0.697]	[0.992, 0.994]	[0.528, 0.536]	[0.738, 0.744]
	FTL	[0.144235, 0.149765]	[0.912, 0.916]	[0.984, 0.986]	[0.850, 0.856]	[0.909, 0.914]

remain similar to the previous node, with benign detection unchanged and a slight decrease in attack detection. The difference in the detection of the unknown attack is only 1.345 %.

As we move to the local model using benign transactions 70 %, the performance begins to decline noticeably. Although detection of benign samples improves slightly, many attack samples are misclassified. An additional increase in benign samples 10 % leads to a significant drop in unknown attack detection from 98.614 % to 83.954 %, a decrease of 14.660 %, in contrast to the previous 1.345 %.

In the following node, with an increase 5 % of benign samples (75 %), there is no improvement in benign detection, while attack detection, particularly for unknown attacks, worsens. From these observations, we can assume a non-linear relationship between benign sample proportions and model performance; beyond certain thresholds, small increases can have substantial negative effects.

For the node with only 10 % attack samples, around half of the attacks are correctly detected, while benign samples are classified with high precision. Overall, the results, particularly from the last nodes, indicate significant room for improvement. Thus, we apply FTL.

Revisiting Table 9, the node with proportions of 50/50 does not seem to greatly benefit from the use of FTL, as the margin of improvement is small. Moreover, it seems that the use of FL achieves a better detection of unknown attacks (99.959 % compared to 99.876 %), although with FTL the detection of all known and unknown attacks is slightly improved. This results in both experiments having the same accuracy (0.992). This suggests that FTL may not be advantageous in more balanced scenarios.

Then, for the node with 60 % benign samples, both the overall detection of the attack and unknown attack detection see slight improvements, but the changes remain marginal (from 98.614 % to 99.872 %), indicating that the model is still too balanced to gain significant benefits from FTL.

For the 70/30 distribution, a notable improvement is observed. FTL improve the detection of unknown attack detection by 10.264 %, increasing from 83.954 % to 94.218 %. The overall accuracy with FTL also improves (0.981), closely resembling the accuracy with FL in the 60/40 partition (0.988).

Increasing benign samples by 5 % from the last node commented (75/25), we find that the FL results are suboptimal. However, FTL improves attack detection, particularly for unknown attacks, which increase from 71.985 % to 89.309 % (a difference of 17.414 %). The accuracy rises from 0.931 with FL to 0.974 with FTL.

In the final node, with a 90/10 traffic distribution, the local model using FL yields lowers results for attack detection, with a recall of 0.532. However, with FTL the detection of all attacks improves by 0.321 (to 0.853), and unknown attacks increase dramatically by 62.614 % (from 19.090 % to 81.704 %). The overall accuracy increases from 0.741 to 0.912, demonstrating substantial improvements in attack identification. These findings suggest that FTL significantly enhances the detection of both known and unknown attacks in more realistic scenarios.

In all nodes, it can be seen that FTL does not perform better in detecting benign traffic, even if the difference is minimal. That is perhaps because the pre-trained model, trained from the Bot-IoT dataset, is not trained with many benign samples. In fact, this might not help with its detection.

Fig. 11 graphically shows that both FL and FTL experience a performance degradation as the data imbalance increases. However, FTL consistently maintains higher accuracy and significantly better unknown attack detection rates, particularly in highly imbalanced scenarios, for example, 75–90% benign traffic.

In conclusion, FL achieves excellent performance in balanced datasets. However, as the proportion of benign samples increases, reflecting real-world conditions, the performance of the model deteriorates, especially for the detection of unknown attacks. This is because local models, after receiving the updated global model, are trained on an abundance of benign samples and few attacks. The implementation

of FTL allows these nodes to enhance their overall accuracy and better detect unknown attacks.

6.1. Scalability and communication overhead analysis

This subsection presents a quantitative analysis of the scalability and communication overhead of the proposed Federated Transfer Learning (FTL) framework as the number of client nodes increases. Although local training is executed in parallel across nodes, global round-time is affected by communication, aggregation, and orchestration processes. Understanding these dynamics is crucial for assessing the practical feasibility of deploying FTL distributed environments in the real world, including 5G networks, where low latency and bandwidth efficiency are key considerations. The results reported in Table 11 provide detailed measurements of local training time, global round duration, and derived communication overhead for different node counts, highlighting the interplay between parallel computation and coordination costs.

To compute the results presented in the Table 11, the following assumptions were applied consistently across all experiments:

- The total number of samples is the same in all experiments and is distributed equally among the nodes.
- Communication overhead is computed as $\text{global round time} - \text{local training time}$, since local training is executed in parallel.
- Model sizes: 10 MB (compressed) and 100 MB (full) for the throughput estimation.

As observed in Table 11, increasing the number of nodes reduces the local training time per iteration, from 11 s for 3 nodes to 4–5 s for 9–11 nodes. This behavior results from the equal distribution of training samples across nodes, which decreases the amount of data each node processes locally. In contrast, the global round time grows moderately, from 2 min 43 s for 3 nodes to 4 min 28 s for 11 nodes, indicating that communication, aggregation, and orchestration overheads dominate the per-round wall-clock time. The jump in global time at 11 nodes suggests that server-side aggregation, protocol overheads, or coordination costs become more significant as the number of participants increases.

A simple per-node bandwidth analysis shows that even for a 100 MB model, the average sustained throughput required per node ranges from 6–11 Mbps, well within typical 5G capabilities. This indicates that network bandwidth is not the limiting factor; the main contributors to the global round are the aggregation, orchestration, and synchronization overheads. Overall, these results indicate that the proposed FTL framework scales reasonably well to 11 nodes, with local training benefiting from the distribution of samples. Future work will focus on optimizations to further improve scalability, such as asynchronous or hierarchical aggregation, gradient/model compression, and leveraging edge or MEC resources, enabling a more comprehensive assessment of convergence, performance, and privacy under realistic 5G conditions.

7. Integration of privacy mechanisms in the proposed FTL framework

Although this study primarily focuses on the algorithmic design and performance evaluation of the proposed Federated Transfer Learning (FTL) framework, data privacy remains an essential aspect for real-world deployment. The modular nature of our architecture facilitates the incorporation of well-established privacy preserving mechanisms on the client side without requiring major structural modifications.

In particular, local Differential Privacy (DP) can be applied during the fine-tuning phase to introduce calibrated noise into gradients or model updates, thus reducing the risk of information leakage from individual data points. Furthermore, secure aggregation protocols can be used to ensure that the central coordinator only accesses aggregated model parameters, maintaining confidentiality of client contributions.

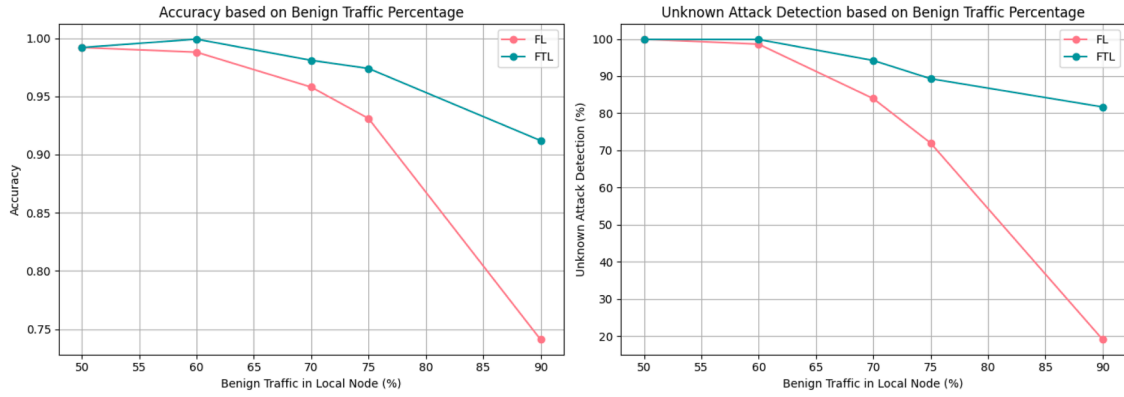


Fig. 11. Evaluation of FL and FTL local models according to benign traffic proportion in local datasets.

Table 11

Analysis of local training and communication costs per round in federated learning: 5 global rounds, 1 local iteration per node.

No. nodes	Local training time (s)	Global round time (s)	Communication overhead (s)	Avg. per-node throughput [Mbps]	
				10 MB model	100 MB model
3	11	163	152	1.05	10.5
5	7	182	175	0.91	9.1
7	5	184	179	0.89	8.9
9	4	190	186	0.86	8.6
11	5	268	263	0.61	6.1

Complementary techniques such as gradient clipping or masking, can further mitigate the exposure of sensitive patterns during communication rounds, while homomorphic encryption offers an alternative for secure computation on encrypted parameters. These privacy-enhancing components can be seamlessly integrated into the current FTL workflow, preserving interoperability between heterogeneous clients. Future work will focus on a systematic evaluation of these mechanisms within our framework to quantify the trade-offs between privacy guarantees, computational overhead, and model performance. The ultimate goal is to achieve a balanced and secure FTL system that maintains high predictive accuracy while ensuring data confidentiality in distributed and heterogeneous environments.

In summary, the application of DP in a FTL setting with multiple nodes requires the combination of DP techniques with the federated training and knowledge transfer workflow. In general terms, the integration of privacy into our model can be described through the following steps:

1. **Pre-trained global model transfer:** The global model, pre-trained in a source domain, is transferred to each node.
2. **Local fine-tuning:** Each node performs fine-tuning on its local dataset.
3. **Differential privacy application:** The local gradients or weights obtained during fine-tuning are processed using DP techniques, including gradient clipping and the addition of calibrated noise.
4. **Server-side aggregation:** The perturbed updates are sent to the central server for aggregation, which can incorporate additional mechanisms such as Secure Aggregation (SA) to further enhance privacy.
5. **Global model update and redistribution:** The global model is updated with the aggregated contributions and redistributed to the nodes.
6. **Iteration until convergence:** This process is repeated iteratively until the global model converges.

8. Conclusions

The course of this paper has focused on developing an IDS suitable specifically for IoT networks that performs attack detection. Malicious

traffic detection has been implemented with Federated Learning (FL) and Federated Transfer Learning (FTL), to monitor whether the addition of Transfer Learning (TL) can help local models improve the detection of unknown attacks, compared to the use of plain FL.

After performing the desired experiments and analyzing the results, we can conclude that applying TL provides a significant model improvement in cases where the imbalance between benign and malicious samples degrades the performance of the local model, i.e., in more realistic scenarios. Using FL with some nodes containing from 50/50 to 60/40 normal/attack proportions provides accuracies of 0.992 and 0.988, respectively. Therefore, in cases where attack samples are frequent in the training dataset, the use of FTL might not significantly improve the detection of known or unknown attacks. In the tests conducted, very similar results were obtained for FTL and FL.

However, most devices typically process a significantly higher number of legitimate transactions than attacks. Therefore, it is important to emphasize that adopting an FTL-based approach can substantially enhance the performance of local models by improving their generalization capabilities. This enhancement benefits not only the detection of known attacks but also the identification of previously unseen ones.

In our experiments, after appropriate fine-tuning, we observed notable improvements in nodes that contain at least 70 % benign samples. The smallest significant increase in the detection rate of unknown attacks was 10.264 % (from 83.954 % to 94.218 %) in nodes with a 70/30 benign-to-malicious distribution. The improvement reached 62.614 % (from 19.090 % to 81.704 %) in the nodes with a 90/10 distribution. In general, these nodes achieved accuracy of 0.981 and 0.912, respectively.

In summary, FTL has proven to be particularly effective in scenarios characterized by node imbalance.

Although the present work has advanced the understanding of FL and FTL in this context, it also opens several promising avenues for future research, as outlined below.

- In this study, weight sharing was used, given the public nature of the dataset and the fact that the features used do not contain sensitive information. However, future work could investigate the use of gradient sharing to enhance privacy within the federated

framework. Furthermore, exploring the integration of differential privacy techniques would further strengthen data protection.

- Since FTL has been applied here to source and target domains sharing the same feature space, future extensions could explore more heterogeneous data partitions. Such experiments would provide insight into more realistic and challenging scenarios.
- As the federated simulations developed in this work were designed to allow full customization of the training process, it would be valuable to compare their performance and flexibility with established frameworks such as NVIDIA FLARE or TensorFlow Federated (TensorFlow, 2019).

CRedit authorship contribution statement

Andrea Bellmunt: Methodology, Software, Writing – original draft preparation; **Beatriz Otero:** Conceptualization, Methodology, Visualization, Investigation, Data curation, Writing – reviewing and editing, Supervision; **Eva Rodríguez:** Conceptualization, Methodology, Investigation, Data curation, Supervision; **Xavier Masip-Bruin:** Funding acquisition, Project administration.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by the Spanish Ministry of Science and Innovation through contracts PID2024-1561500B-IOO and EQC2024-008344-P, as well as by the Generalitat de Catalunya under grant 2021-SGR-00326. Furthermore, research that contributed to these results received funding from the European Union's Horizon 2020 research and innovation programme under the HE 6G LEADER (101192080), HE HORSE (101096342), HE MARE (101191436), and HE VITAMIN-V (101093062) projects.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ..., & Zheng, X. (2016). TensorFlow: Large-scale machine learning on heterogeneous distributed systems. *abs/1603.04467*.
- Agrawal, S., Sarkar, S., Aouedi, O., Yenduri, G., Piamrat, K., Alazab, M., Bhattacharya, S., Maddikunta, P. K. R., & Gadekallu, T. R. (2022). Federated learning for intrusion detection system: Concepts, challenges and future directions. *Computer Communications*, 195, 346–361. <https://doi.org/10.1016/j.comcom.2022.09.012>
- Alalyan, F., Bousalem, B., Jaafar, W., & Langar, R. (2024). Secure peer-to-peer federated learning for efficient cyberattacks detection in 5G and beyond networks. In *ICC 2024-IEEE international Conference on Communications* (pp. 1752–1757). Denver, USA: IEEE. <https://doi.org/10.1109/ICC51166.2024.10622894>
- Alharbi, A. A. (2024). Federated transfer learning for attack detection for internet of medical things. *International Journal of Information Security*, 23(1), 81–100. <https://doi.org/10.1007/s10207-023-00805-9>
- Bellmunt, A. (2024). Federated transfer learning-based intrusion detection system in 5G networks. Bachelor's thesis Barcelona, Spain Universitat Politècnica de Catalunya, Facultat d'Informàtica de Barcelona.
- Bellmunt, A. (2025). FTL-based IDS in 5G networks. <https://github.com/andreabellmunt/FTL-based-IDS-in-5G-networks>. Accessed: 2025-10-30.
- Blanchard, P., El Mhamdi, E. M., Guerraoui, R., & Stainer, J. (2017). Machine learning with adversaries: Byzantine tolerant gradient descent. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems* (pp. 1–11). Curran Associates, Inc. (vol. 30).
- Blika, A., Pamos, S., Doukas, G., Lamprou, V., Pelekis, S., Kontoulis, M., Ntanos, C., & Askounis, D. (2025). Federated learning for enhanced cybersecurity and trustworthiness in 5G and 6G networks: A comprehensive survey. *IEEE Open Journal of the Communications Society*, 6, 3094–3130. <https://doi.org/10.1109/OJCOMS.2024.3449563>
- Chettri, L., & Bera, R. (2020). A comprehensive survey on internet of things (IoT) toward 5G wireless systems. *IEEE Internet of Things Journal*, 7(1), 16–32. <https://doi.org/10.1109/JIOT.2019.2948888>
- Djaidja, T. E. T., Brik, B., Boualouache, A., Senouci, S. M., & Ghamri-Doudane, Y. (2024). Federated learning for 5G and beyond, a blessing and a curse-an experimental study on intrusion detection systems. *Computers & Security*, 139, 103707. <https://doi.org/10.1016/j.cose.2024.103707>
- Fan, Y., Li, Y., Zhan, M., Cui, H., & Zhang, Y. (2020). IoTdefender: A federated transfer learning intrusion detection framework for 5G IoT. In *2020 IEEE 14th international conference on big data science and engineering (BigDataSE)* (pp. 88–95). IEEE. <https://doi.org/10.1109/BigDataSE50710.2020.00020>
- Gupta, A., & Jha, R. K. (2015). A survey of 5G network: Architecture and emerging technologies. *IEEE Access*, 3, 1206–1232. <https://doi.org/10.1109/ACCESS.2015.2461602>
- Harris, C. R., Millman, K. J., & van der Walt, S. J. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Iman, M., Arabnia, H. R., & Rasheed, K. (2023). A review of deep transfer learning and recent advancements. *Technologies*, 11(2). <https://doi.org/10.3390/technologies11020040>
- Jamal, M. H., Alazeb, A., Bakhsh, S. A., Boulila, W., Shah, S. A., Khattak, A. A., & Khan, M. S. (2025). Optimizing fire safety: Reducing false alarms using advanced machine learning techniques. In A. Koubaa, A. B. Mnaouer, W. Boulila, & S. Raghuay (Eds.), *International conference on smart systems and emerging technologies* (pp. 409–420). Cham: Springer Nature Switzerland.
- Jamal, M. H., Khan, M. A., Ullah, S., Alshehri, M. S., Almakdi, S., Rashid, U., Alazeb, A., & Ahmad, J. (2023a). Multi-step attack detection in industrial networks using a hybrid deep learning architecture. *Mathematical Biosciences and Engineering*, 20(8), 13824–13848. <https://doi.org/10.3934/mbe.2023615>
- Jamal, M. H., Naz, N., Khattak, M. A. K., Saeed, F., Altamimi, S. N., & Qasem, S. N. (2023b). A comparison of re-sampling techniques for detection of multi-step attacks on deep learning models. *IEEE Access*, 11, 127446–127457. <https://doi.org/10.1109/ACCESS.2023.3332512>
- Koroniotis, N. (2020). Designing an effective network forensic framework for the investigation of botnets in the internet of things. Ph.D. thesis. UNSW Sydney.
- Koroniotis, N., & Moustafa, N. (2020). Enhancing network forensics with particle swarm and deep learning: The particle deep framework. 2005.00722 CoRR.
- Koroniotis, N., Moustafa, N., Schilero, F., Gauravaram, P., & Janicke, H. (2020a). A holistic review of cybersecurity and reliability perspectives in smart airports. *IEEE Access*, 8, 209802–209834. <https://doi.org/10.1109/ACCESS.2020.3036728>
- Koroniotis, N., Moustafa, N., & Sitnikova, E. (2020b). A new network forensic framework based on deep learning for internet of things networks: A particle deep framework. *Future Generation Computer Systems*, 110, 91–106. <https://doi.org/10.1016/j.future.2020.03.042>
- Koroniotis, N., Moustafa, N., Sitnikova, E., & Slay, J. (2018a). Towards developing network forensic mechanism for botnet activities in the IoT based on machine learning techniques. In J. Hu, I. Khalil, Z. Tari, & S. Wen (Eds.), *Mobile networks and management* (pp. 30–44). Springer International Publishing.
- Koroniotis, N., Moustafa, N., Sitnikova, E., & Turnbull, B. (2019). Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset. *Future Generation Computer Systems*, 100, 779–796 <https://doi.org/10.1016/j.future.2019.05.041>
- Lee, J., Solat, F., Kim, T. Y., & Poor, H. V. (2024). Federated learning-empowered mobile network management for 5G and beyond networks: From access to core. *IEEE Communications Surveys & Tutorials*, 23, 2176–2212. <https://doi.org/10.1109/COMST.2024.3352910>
- Li, S., Ngai, E. C.-H., & Voigt, T. (2024). An experimental study of Byzantine-robust aggregation schemes in federated learning. *IEEE Transactions on Big Data*, 10(6), 975–988. <https://doi.org/10.1109/TBDDATA.2023.3237397>
- Madakam, S., Ramaswamy, R., & Tripathi, S. (2015). Internet of things (IoT): A literature review. *Journal of Computer and Communications*, 3, 164–173. <https://doi.org/10.4236/jcc.2015.35021>
- McKinney, W. et al. (2010). Data structures for statistical computing in python. In S. van der Walt, & J. Millman (Eds.), *Proceedings of the 9th python in science conference* (pp. 51–56). (445). <https://doi.org/10.25080/Majora-92b1922-00a>
- McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. y. (2017). Communication-efficient learning of deep networks from decentralized data. In A. Singh, & J. Zhu (Eds.), *Proceedings of the 20th international conference on artificial intelligence and statistics* (pp. 1273–1282). PMLR (vol. 54).
- McMahan, B., & Ramage, D. (2017). Federated learning: Collaborative machine learning without centralized training. Google Research Blog. Accessed: 2025-10-29.
- Mohammadpour, L., Ling, T. C., Liew, C. S., & Aryanfar, A. (2022). A survey of CNN-based network intrusion detection. *Applied Sciences*, 12(16). <https://doi.org/10.3390/app12168162>
- Moustafa, N., Creech, G., & Slay, J. (2017). Big data analytics for intrusion detection system: Statistical decision-making using finite Dirichlet mixture models. In I. Palomares Carrascosa, H. K. Kaluturage, & Y. Huang (Eds.), *Data analytics and decision support for cybersecurity: Trends, Methodologies and Applications*, pp. 127–156). Springer International Publishing. https://doi.org/10.1007/978-3-319-59439-2_5
- Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 military communications and information systems conference (MILCIS)* (pp. 1–6). <https://doi.org/10.1109/MILCIS.2015.7348942>
- Moustafa, N., & Slay, J. (2016). The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Information Security Journal: A Global Perspective*, 25(1–3), 18–31. <https://doi.org/10.1080/19393555.2015.1125974>

- Moustafa, N., Slay, J., & Creech, G. (2019). Novel geometric area analysis technique for anomaly detection using trapezoidal area estimation on large-scale networks. *IEEE Transactions on Big Data*, 5(4), 481–494. <https://doi.org/10.1109/TBDATA.2017.2715166>
- Otoun, Y., Chamola, V., & Nayak, A. (2022). Federated and transfer learning-empowered intrusion detection for IoT applications. *IEEE Internet of Things Magazine*, 5(3), 50–54. <https://doi.org/10.1109/IOTM.001.2200048>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Pillutla, K., Kakade, S. M., & Harchaoui, Z. (2022). Robust aggregation for federated learning. *IEEE Transactions on Signal Processing*, 70, 1142–1154. <https://doi.org/10.1109/TSP.2022.3153135>
- Rahman, K. M. J., Ahmed, F., Akhter, N., Hasan, M., Amin, R., Aziz, K. E., Islam, A. K. M. M., Mukta, M. S. H., & Islam, A. K. M. N. (2021). Challenges, applications and design aspects of federated learning: A survey. *IEEE Access*, 9, 124682–124700. <https://doi.org/10.1109/ACCESS.2021.3111118>
- Rodríguez, E., Otero, B., Gutiérrez, N., & Canal, R. (2021). A survey of deep learning techniques for cybersecurity in mobile networks. *IEEE Communications Surveys & Tutorials*, 23(3), 1920–1955. <https://doi.org/10.1109/COMST.2021.3086296>
- Rodríguez, E., Valls, P., Otero, B., Costa, J. J., Verdú, J., Pajuelo, M. A., & Canal, R. (2022). Transfer-learning-based intrusion detection framework in IoT networks. *Sensors*, 22(15), 1–17. <https://doi.org/10.3390/s22155621>
- Samarakoon, S., Siriwardhana, Y., Porambage, P., Liyanage, M., Chang, S.-Y., Kim, J., Kim, J., Ylianttila, M., 2022. 5G-NIDD: A comprehensive network intrusion detection dataset generated over 5G wireless network. *IEEE Dataport* <https://doi.org/10.21227/xtep-hv36>.
- Sarhan, M., Layeghy, S., Moustafa, N., & Portmann, M. (2021). NetFlow datasets for machine learning-based network intrusion detection systems. In *Big data technologies and applications*, pp. 117–135. Springer International Publishing. https://doi.org/10.1007/978-3-030-72802-1_9
- Sharafaldin, I., Habibi Lashkari, A., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th international conference on information systems security and privacy - ICISSP* (pp. 108–116). SciTePress. <https://doi.org/10.5220/0006639801080116>
- Tabian, I., Fu, H., & Sharif Khodaei, Z. (2019). A convolutional neural network for impact detection and characterization of complex composite structures. *Sensors*, 19(22). <https://doi.org/10.3390/s19224933>
- TensorFlow (2019). Tensorflow federated: Machine learning on decentralized data. <https://www.tensorflow.org/federated>. Accessed: 2025-10-29.
- Verma, P., Bharot, N., Breslin, J. G., O'Shea, D., Vidyarthi, A., & Gupta, D. (2023). Zero-day guardian: A dual model enabled federated learning framework for handling zero-day attacks in 5G enabled IIoT. *IEEE Transactions on Consumer Electronics*, 70, 3856–3866. <https://doi.org/10.1109/TCE.2023.3335385>
- Xia, L., Zheng, P., Li, J., Tang, W., & Zhang, X. (2022). Privacy-preserving gradient boosting tree: Vertical federated learning for collaborative bearing fault diagnosis. *IET Collaborative Intelligent Manufacturing*, 4(3), 208–219. <https://doi.org/10.1049/cim2.12057>
- Zhang, J., Luo, C., Carpenter, M., & Min, G. (2022). Federated learning for distributed IIoT intrusion detection using transfer approaches. *IEEE Transactions on Industrial Informatics*, 19(7), 8159–8169. <https://doi.org/10.1109/TII.2022.3216575>